README

AI Agents Ultimate Setup Wizard

This fully automated wizard (`scripts/setup-agents-wizard.sh`) installs and configures **five leading AI coding agents** on your system:

- **Claude Code** (Anthropic)
- **Kimi** (Moonshot AI)
- **Opencode** (Open-source CLI agent)
- **MiMo Code** (MiMo AI)
- **Qwen Code** (Alibaba)

It also equips every agent it can with the high-efficiency **MCP (Model Context Protocol) server Lumen** – and, for **Kimi** and **Qwen Code**, **CodeGraph** too – plus a shared stack of performance optimizers (`ashlr`, `WOZCODE`, `Glyphdown`, `SpecKit`, `SuperSpec`).

> **A shell script cannot do everything.** Claude Code slash commands run *inside* Claude Code, and privileged host changes need your `sudo`. Rather than skipping those silently and printing a green summary anyway, the wizard **detects** them, prints an **ACTION REQUIRED** section, writes `MANUAL-STEPS.md` to your project root, and pauses for Enter on a terminal. See ACTION REQUIRED. Everything the wizard *does* change is recorded and can be undone — see Safety and reversibility.

Documentation

This README is the entry point. The companion documents go deeper:

Document	What it covers
USER_GUIDE.md	Task-oriented walkthrough: quick start, what each of the nine steps prints, how to verify the result, first Lumen index and search, what to do if something goes wrong.
MANUAL.md	Exhaustive reference: every function, constant, file created or modified, exit code, environment variable, library mode, and the Lumen CLI.
SAFETY-AND-ROLLBACK.md	**The undo story.** The transactional backup manifest, the full `scripts/rollback-agents-wizard.sh` CLI, per-component rollback recipes, how to reverse a rollback, manual uninstall per component, and exactly what is and is not reversible.
FAQ.md	Why the design is what it is: no npm Lumen, wrapper instead of symlink, `stdio` vs `serve`, re-run safety, index storage, the telemetry opt-out, complete uninstall.
TROUBLESHOOTING.md	Symptom-first fixes: launcher not found, stuck indexing, empty search results, unreachable embedding backend, duplicate `PATH` entries, restoring from a backup session.
ARCHITECTURE.md	Diagrams and rationale: end-to-end flow, Lumen resolution sequence, PATH wiring, files touched, test-coverage map, indexing state machine.
OPERATIONAL-SCRIPTS.md	**The three scripts that run *after* installation**: the ollama Vulkan remediation, the resilient reindexer, and the index doctor.

Document	What it covers
ACTION-REQUIRED.md	Every flag, every exit code, and what each one will *not* do. The manual-step system: what the wizard genuinely cannot do, how it detects each pending step (`plugins/cache/` vs `plugins/marketplaces/`), `MANUAL-STEPS.md`, the Enter pause and `WIZARD_NONINTERACTIVE`, ashlr-as-a-plugin and the genome.
OLLAMA-REMEDIATION.md	The operator runbook for an ollama backend that corrupts embeddings on a Vulkan iGPU. **Status on this host: resolved** — the record of what was applied and how it was verified.

Historical records — read, do not edit. These are dated investigations kept as evidence. Where they disagree with the list above, the list above wins.

Document	What it is
INDEX-CORRUPTION-RECONCILIATION.md	**The settled verdict on the index.** 758 stale-duplicate vectors across 55 files; why the batch total, not the chunk size, was the trigger. Start here for anything about index trustworthiness.
LUMEN-INDEX-INTEGRITY.md	An earlier full audit that concluded the index was `TRUSTWORTHY`. Its measurements are correct and reproduce exactly; **its conclusion is superseded** — see Why “no NaN” did not mean “no corruption”.
OLLAMA-NAN-WEDGE.md	The original evidence and harness for the loud failure

Document	What it is
	modes (HTTP 500 `NaN`, all-zero vectors, the wedged runner).
`INDEPENDENT-VERIFICATION.md`	An adversarial replay of this documentation's claims against the code.
`INDEPENDENT-VERIFICATION-2.md`	A second adversarial pass: 15 claims tested, 1 refuted, 2 verified with caveats, 4 distinct defects found.
`DOC-AUDIT.md`	A documentation-vs-code consistency audit.
`CODEGRAPH-UPGRADE.md`	CodeGraph version notes.

Script	What it is
`scripts/setup-agents-wizard.sh`	The wizard.
`scripts/rollback-agents-wizard.sh`	Undoes any wizard run, wholesale or one component at a time. See Safety and reversibility.
`scripts/test-setup-agents-wizard.sh`	The test suite; `--no-live` skips the network/daemon checks.
`scripts/ollama-vulkan-remediation.sh`	`--check` (default, read-only) · `--apply` · `--verify` · `--rollback`. Takes the GPU out of ollama's embedding path. Reference.
`scripts/lumen-reindex.sh`	`[path] [--force] [--allow-gpu]`. Resilient reindex that **refuses to run on a Vulkan backend**. Reference.
`scripts/lumen-index-doctor.sh`	`[path]`. Read-only corruption check; **exit 1 means corruption found**. Reference.
`scripts/audit-hardcoded-paths.sh`	`[repo-path]` / `--list`. Fails the build on machine-specific absolute paths. Reference.

The root `CLAUDE.md` / `AGENTS.md` / `QWEN.md` / `GEMINI.md` files

The wizard does not create them, and nothing here reads or writes them. They are the repository's **governance carriers**: each opens with a `## INHERITED FROM` pointer into the `submodules/constitution/` git submodule, and together with the root `Constitution.md` they are what the agents this wizard installs pick up as project rules.

If you want to know what they are, why they are pointers instead of `@imports`, why all four must stay byte-identical below their header, or how to run the two verifiers that check them, start at `../constitution-adoption/README.md`.

Prerequisites

- **System**: Linux (Debian/RedHat) or macOS
- **Core tools**: `git`, `curl`, `Node.js` (v18+), `npm`
- **Internet**: Required to download packages and clone repositories

The script automatically installs `jq` (for JSON editing) if missing, and installs **Ollama** – Lumen's local embedding backend – in Step 3 if it is not already present.

Running the Script

The script already lives in this repository at `scripts/setup-agents-wizard.sh`.

1. **Make it executable** (once):

   ```
   chmod +x scripts/setup-agents-wizard.sh
   ```

2. **Execute** from anywhere in the project – it auto-detects the project root (the parent of `scripts/`):

   ```
   ./scripts/setup-agents-wizard.sh
   ```

   > The script is **idempotent** – you can run it multiple times safely. It is non-interactive throughout, with exactly **one** exception: at the very end it pauses for Enter so the ACTION REQUIRED list cannot scroll past unread. That pause is skipped when stdin is not a

> terminal (pipes, CI) or when `WIZARD_NONINTERACTIVE=1` is set. It *can* also ask for a `sudo` password when installing `jq`, installing Ollama, or enabling the `ollama` service — that prompt comes from `sudo`, not from the script — so run it from a terminal you are watching.

The Nine Steps

Steps 1–7 and 9 always run. **Step 8 is opt-in** and only appears when you set `WIZARD_INDEX_PROJECT=1`.

Step	Header printed	What happens
1	`Step 1: Checking System Prerequisites`	Verifies `git`, `curl`, `node`, `npm`. Exits `1` if any is missing. Installs `jq` when needed.
2	`Step 2: Installing / Updating Global CLI Tools`	Bun, then **only the tools that are missing**, by verified package name: `@colbymchenry/codegraph`, `glyphdown`, `@qwen-code/qwen-code`, and `specify` via `uv tool install specify-cli`. Kimi, Opencode and MiMo Code are **detected, not installed** — they ship their own installers. Then `ashlr-plugin` and optional WOZCODE.
3	`Step 3: Lumen Semantic Search Setup`	First disables usage analytics across the installed tooling (see Telemetry opt-out), then installs the `~/.local/bin/lumen` wrapper and bash completions, writes the managed `~/.bashrc` and `~/.bash_profile` PATH blocks, ensures Ollama plus the embedding model, and verifies all of it.
4		

Step	Header printed	What happens
	`Step 4: Verifying CLI Availability`	Prints ✓ / ✗ for the **eight** CLIs: `lumen`, `codegraph`, `glyphdown`, `specify`, `kimi`, `opencode`, `mimo`, `qwen`. (`qwen` is the binary; `@qwen-code/qwen-code` is the package that provides it.) **`ashlr` is deliberately not in this list** — it is a Claude Code plugin with no binary, reported in its own summary section instead.
5	`Step 5: Configuring MCP Servers for Each Agent`	Extends each agent's **real** config in place: `claude mcp add-json` for Claude Code, a `jq` mirror into `~/.claude.json` (which MiMo and friends inherit from), `~/.kimi-code/mcp.json`, `~/.config/opencode/opencode.json`, `~/.qwen/settings.json`. Clones the Claude marketplace plugins and adds the Glyphdown `PreToolUse`/`PostToolUse` hooks when `glyphdown` is installed and `WIZARD_SKIP_GLYPHDOWN_HOOK` is unset. MiMo Code is **probed** with `mimo mcp list`, not guessed at.
6	`Step 6: Project-Level Setup`	Creates `submodules/`, initializes SpecKit, checks out SuperSpec, registers it as a SpecKit dev extension.
7	`Step 7: Ollama Concurrency Tuning (host-specific)`	Delegates to `scripts/ollama-tune.sh`: it detects how ollama is managed on **this** host and computes a concurrency figure from this host's CPU/RAM/model

Step	Header printed	What happens
		facts. The wizard shows you the detection and the recommendation, and **stops there** — applying restarts the ollama service, which aborts every in-flight embedding request. The exact commands the tuner generated **for this host** go into ACTION REQUIRED; nothing generic is ever printed in their place. Opt in with `WIZARD_TUNE_OLLAMA=1` and it will still ask, and will still refuse while an indexer is running or while it cannot tell. The tuner's exit code is a **three-valued verdict** — `0` fine, `1` action required (there IS a recommendation), `2` could not determine — and all three are preserved; `1` is never mistaken for a broken tuner. If the tuner script or ollama is absent, the step says so and the wizard continues.
8	`Step 8: Indexing This Project`	**Opt-in — only with `WIZARD_INDEX_PROJECT=1`.** Refreshes the CodeGraph index for the project root (`codegraph sync` when `.codegraph/codegraph.db` already exists, `codegraph init` when it does not — never `codegraph index`, which discards the existing database), then runs `lumen index "$PROJECT_ROOT"`. Without the variable the

Step	Header printed	What happens
		wizard prints `Skipping project indexing (set WIZARD_INDEX_PROJECT=1 to build indexes).` and does nothing.
9	`Step 9: Provider-Side CI Verification`	Read-only. Delegates to `scripts/verify-provider-ci.sh`, which enumerates the owned repositories from this checkout's own remotes and **queries the provider** for what no file-level check can see: Pages `build_type`, provider-generated `pages build and deployment` runs, branch protection / required checks, Actions enablement. Three-valued and kept that way: **0** nothing found, **1** confirmed (→ ACTION REQUIRED, carrying the verifier's own findings, because changing a provider setting means opening a provider UI), **2** could not determine — reported as **UNVERIFIED, never as a pass**. Missing script or missing `gh` is likewise UNVERIFIED, not clean, and the wizard continues.

After the final summary the wizard prints an **ACTION REQUIRED** section — the steps a script genuinely cannot perform — and, on a terminal, waits for Enter. See ACTION REQUIRED.

For the full narration of each step – including the exact console output – see `USER_GUIDE.md`.

ACTION REQUIRED — the steps only you can do

Two kinds of work are outside a shell script's reach: **Claude Code slash commands** (they run inside a Claude Code session; there is no CLI equivalent) and **privileged host changes** (they need your `sudo`, and restarting a system service is not an installer's decision to make quietly).

A wizard that skips those silently prints a green summary for work that was never done. This one does the opposite:

```
======================================
 ACTION REQUIRED — 3 step(s) only you can do
======================================
A shell script cannot run Claude Code slash commands or use your
sudo.
These are NOT done. Nothing above claims they are.

1. Activate the ashlr plugin inside Claude Code
     /plugin marketplace add ashlrai/ashlr-plugin
     /plugin install ashlr@ashlr-marketplace
     /reload-plugins
     /ashlr:ashlr-status          # confirm it is live
...
    Saved to /path/to/project/MANUAL-STEPS.md so you can come back
to it.

Read the 3 step(s) above. Press Enter to continue...
```

Detection is stateful, and that is the whole trick. Claude Code keeps two directories that mean different things:

Path	Written by	Means
`<claude-dir>/plugins/cache/<mkt>/`	the vendor installer the wizard ran	the bits are on disk — **nothing is wired in**
`<claude-dir>/plugins/marketplaces/<mkt>/`	Claude Code, when *you* run `/plugin marketplace add`	**a human actually did it**

So the wizard can tell "I downloaded it" from "you activated it" without a state file of its own. Do the step, re-run the wizard, and the entry disappears — because the evidence of having done it is a directory that

now exists. Test `A50` guards the `marketplaces/` probe specifically; checking only the cache would report the plugin as done the moment it was downloaded.

The steps it can raise: activate the ashlr plugin · initialise the ashlr genome · apply the ollama Vulkan remediation · supply a WOZCODE installer · re-enable the skipped glyphdown hook · build the Lumen index · refresh `PATH` (always). Each is written to `<project-root>/MANUAL-STEPS.md`, overwritten every run.

```
WIZARD_NONINTERACTIVE=1 ./scripts/setup-agents-wizard.sh
        # print the list, skip the Enter pause
```

The pause needs all three of: stdin is a terminal, `WIZARD_NONINTERACTIVE` unset, at least one step. In a pipe or a CI job it never blocks. This is the wizard's **only** blocking prompt.

Full reference — every condition, the exact command text, the `MANUAL-STEPS.md` format, and why `ashlr` has no binary: `ACTION-REQUIRED.md`.

What Gets Installed

Global System Tools

Tool	Purpose	Installation Source
Bun	JavaScript runtime	Official `curl` installer
Lumen	MCP semantic-search server + CLI	**Not npm.** Claude Code plugin binary + wrapper on `PATH` (see below)
Ollama	Local embedding backend Lumen indexes with	Official `curl` installer (Linux) / `brew` (macOS)
CodeGraph	MCP code-graph server	`npm` (`@colbymchenry/codegraph`)
Glyphdown	Lossless I/O token compressor	`npm` (`glyphdown`)
SpecKit	Spec-driven development CLI	`uv tool install specify-cli` (a Python tool, **not** an npm package)

Tool	Purpose	Installation Source
ashlr-plugin	Token-efficient MCP tools	Official `curl` installer (needs `bun`). **A Claude Code plugin, not a CLI** — it clones into `~/.claude/plugins/cache/ashlr-marketplace/ashlr/` and creates no binary. `command -v ashlr` never succeeds *by design*; three slash commands finish the job — see ACTION REQUIRED
WOZCODE	Batching toolset	Via `WOZCODE_INSTALL_CMD` env var (optional)

How Lumen Is Actually Installed

Lumen is **not** an npm package. It ships as a **Claude Code plugin binary**:

```
~/.claude-shared/plugins/cache/<marketplace>/lumen/<version>/bin/lumen-<os>-<arch>
```

That `<version>` component changes on every plugin update, so a symlink or a hardcoded `PATH` entry would silently break at the next upgrade. Instead, Step 3 installs a **version-agnostic wrapper** at `~/.local/bin/lumen` that resolves the newest installed version **at call time** using `sort -V` – a lexical sort would rank `0.0.9` above `0.0.100`. If nothing is found under `~/.claude-shared/...`, the wrapper falls back to `~/.claude*/plugins/cache/*/lumen/*/bin/`. Setting `LUMEN_BIN=/path/to/binary` overrides resolution entirely.

The wrapper is deliberately reachable from **login and non-interactive** shells too: `~/.bashrc` returns early when non-interactive, so the wizard also writes a `~/.local/bin` guard into `~/.bash_profile`. Without it, `ssh host '<cmd>'`, cron jobs and systemd user units would never see `lumen`.

AI Agent CLIs

Agent	How it is obtained
Claude Code	*(built into the IDE/extension – the wizard only registers the MCP server)*
Kimi	Vendor installer, `~/.kimi-code/bin` – https://kimi.moonshot.cn. **Detected, never npm-installed**
Opencode	Vendor installer, `~/.opencode/bin` – https://opencode.ai. **Detected, never npm-installed**
MiMo Code	Vendor installer, `~/.mimocode/bin` – https://github.com/XiaomiMiMo. **Detected, never npm-installed**
Qwen Code	`npm install -g @qwen-code/qwen-code`, only when `qwen` is missing. The package is `@qwen-code/qwen-code`; the **binary it provides is `qwen`**, which is what the wizard probes

The bare npm names `kimi`, `opencode` and `mimo` are **not** these agents: `opencode` returns 404, while `kimi` is a state-animation library and `mimo` a minimal mobile build tool. Installing by guessed name pollutes the global npm prefix with unrelated software, so the wizard detects these three and points at their vendor instead.

MCP Configuration for Every Agent

The wizard extends each agent's **real** configuration in place. These paths were verified on disk:

Agent	Where Lumen is registered	Schema
Claude Code	`claude mcp add-json lumen … -s user` — writes into the **active `CLAUDE_CONFIG_DIR`**	n/a – managed by the CLI
(inheritors)		

Agent	Where Lumen is registered	Schema
	`~/.claude.json`, mirrored with `jq` when that file exists	`.mcpServers.lumen = {"type":"stdio","command":<wrapper>,"args":["stdio"]}`
Kimi	`~/.kimi-code/mcp.json`	`.mcpServers`
Opencode	`~/.config/opencode/opencode.json`	`.mcp` — **not** `.mcpServers`
MiMo Code	*(nothing MiMo-specific)* — it **inherits** from `~/.claude.json`, which the row above fills in	verified with `mimo mcp list`, not assumed
Qwen Code	`~/.qwen/settings.json`	`.mcpServers`

> **Why the `~/.claude.json` mirror exists.** `claude mcp add-json -s user` writes into the config directory the session is actually using — `$CLAUDE_CONFIG_DIR` when set, `~/.claude` otherwise — which is **not necessarily `~/.claude.json`**. Tools that inherit from the default file (MiMo labels its servers `claude:~/.claude.json`) would then stay blind to Lumen even though Claude Code itself had it. The wizard therefore `jq`-merges the same entry into `~/.claude.json` directly, backing it up under component `claude` first. Tests `A44`, `A45` and `A46`.
>
> **An earlier revision of this README claimed MiMo Code "has no documented MCP config file" and printed a manual step for it. That claim was false** — MiMo never needed one. Verify with `mimo mcp list | grep lumen`.

Kimi and Qwen Code receive both servers, in the `.mcpServers` schema:

```
{
  "mcpServers": {
    "lumen": {
      "command": "/home/you/.local/bin/lumen",
      "args": ["stdio"]
    },
    "codegraph": {
      "command": "codegraph",
```

```
      "args": ["serve"]
    }
  }
}
```

Opencode uses a different schema, so it gets its own shape:

```
{
  "mcp": {
    "lumen": {
      "type": "local",
      "command": ["/home/you/.local/bin/lumen", "stdio"],
      "enabled": true
    }
  }
}
```

Claude Code is configured through its own CLI rather than by hand-editing `~/.claude.json`, which also holds session and project state:

```
claude mcp add-json lumen '{"command":"/home/you/.local/bin/
        lumen","args":["stdio"]}' -s user
```

Three details matter here:

- **`"args": ["stdio"]`** – `stdio` is Lumen's MCP subcommand. There is **no** `serve` subcommand; it fails with `Error: unknown command "serve" for "lumen"`.
- **`"command"` is an absolute, expanded path** (`~/.local/bin/lumen` written out in full). Agents spawn MCP servers from non-interactive shells where `~/.local/bin` is frequently absent from `PATH`, so a bare `"lumen"` would fail to launch.
- **Nothing is invented.** An earlier revision wrote `~/.claude/mcp.json`, `~/.kimi/config.json`, `~/.opencode/config.json`, `~/.mimo/config.json` and `~/.qwen-code/config.json` — orphan files that no agent ever reads, while the summary still printed a success tick for each. If those files exist on your machine, they are leftovers and can be deleted.

Every merge is done with `jq`, so pre-existing keys and MCP servers in those config files are preserved and re-running does not duplicate entries.

Telemetry Opt-Out

Step 3 opens with `configure_telemetry_optout`, which turns usage analytics **off** for the tooling this wizard installs. It is on by default; set `WIZARD_KEEP_TELEMETRY=1` to leave every setting exactly as it was.

Every knob below was verified against the installed package rather than guessed:

What	Where	Effect
`DO_NOT_TRACK=1`	A **third** managed block in `~/.bashrc`, delimited by `# >>> telemetry opt-out (managed by Claude Code) >>>`	The cross-tool convention (https://consoledonottrack.com); `@colbymchenry/codegraph` reads it
`CODEGRAPH_TELEMETRY=0`	Same `~/.bashrc` block	CodeGraph's own switch
`codegraph telemetry off`	Run once, when `codegraph` is on `PATH`	Disables CodeGraph telemetry and deletes its buffered data (state lands in `~/.codegraph/telemetry.json`)
`.usageStatisticsEnabled = false`	`~/.qwen/settings.json`, via `jq`	Read by `@qwen-code/qwen-code`. Only touched when the file already exists and `jq` is available; the file is backed up under component `qwen` first

Both variables are also exported into the wizard's own process, so the rest of the run is covered too.

> **Lumen ships no telemetry at all.** Its binary was checked and contains zero analytics strings, so there is nothing to disable for it — the summary says so explicitly rather than claiming a switch that does not exist.

The final summary gains a `Telemetry / analytics:` section reporting each of these, or `left untouched on request (WIZARD_KEEP_TELEMETRY)`.

Project-Level Setup

- Creates `submodules/` directory at your project root.
- Initializes **SpecKit** (`.specify/` or `specs/` folder).
- Checks out **SuperSpec** into `submodules/superspec` – reusing an existing checkout, initializing it with `git submodule update --init --recursive` when it is a registered-but-missing submodule, and cloning only as a last resort.
- Registers SuperSpec as a dev extension for SpecKit (`specify extension add`).

Project Indexing (Step 8, opt-in)

Installing the indexers does not index anything. Both passes are opt-in because a first Lumen pass on a large repository is CPU-bound on the embedding backend and can run for hours.

```
WIZARD_INDEX_PROJECT=1 ./scripts/setup-agents-wizard.sh
```

With that set, Step 8 runs against `PROJECT_ROOT`:

- **CodeGraph** — `codegraph sync "$PROJECT_ROOT"` when `.codegraph/codegraph.db` already exists, otherwise `codegraph init "$PROJECT_ROOT"` (which creates `.codegraph/` in the project root). The wizard never runs `codegraph index`: that command discards the existing database before rebuilding.
- **Lumen** — `lumen index "$PROJECT_ROOT"`, incremental.

Either half is skipped with a warning if its CLI is missing. Neither the CodeGraph database nor the Lumen index is recorded in the rollback manifest — remove them with `rm -rf .codegraph` and `lumen purge "$PROJECT_ROOT"`.

Why “no NaN” did not mean “no corruption”

This is the most important thing in this documentation set. If you read one section, read this one.

On this host, `ollama` was serving the embedding model with `library=Vulkan` on an Intel iGPU. A large embedding batch trips an i915 fence timeout; the kernel abandons the dispatch and **ollama reads the result buffer anyway**. There are four failure modes, and they get quieter as they get worse:

#	What comes back	HTTP	Caught by a per-vector check?
1	`{"error":"…unsupported value: NaN"}`	500	Yes — loud
2	An all-zero vector	**200**	Yes — a zero-norm test sees it
3	The runner wedges: every later request returns `NaN`	500	Yes, but usually blamed on the caller
4	**A repeated STALE vector** — well-formed, 768 dims, L2 norm 1.000000	**200**	**No. Invisible to NaN, Inf, all-zero and norm checks alike.**

Mode 4 wrote **758 identical vectors covering 695 distinct texts across 55 files** into this project’s index. Searches kept working. Every health check kept passing.

The audit that got it wrong

LUMEN-INDEX-INTEGRITY.md examined that index and concluded it was **`TRUSTWORTHY`**. Every number in it is correct and reproduces exactly: 0 NaN, 0 Inf, 0 all-zero, 0 wrong-dimension, every norm inside 0.999999–1.000001.

The conclusion did not follow from the measurements. All four tests are *per-vector*, and a stale-but-well-formed vector passes all four by construction. The one check that would have caught it — aggregate distinctness — was run on blocks 1–4 only (4,096 of 35,717 vectors). The corruption lived in blocks 28–29.

> **A vector can be perfectly well-formed and still be the wrong vector.** Health is not a property you can establish one vector at a time. *N* distinct texts must yield *N* distinct vectors, and nothing short of that comparison can tell you whether they did.

That report is kept **unedited**, as a historical record of how a rigorous audit reached a wrong conclusion. **The settled verdict is `INDEX-CORRUPTION-RECONCILIATION.md`** — go there for anything about whether an index can be trusted. It also falsifies the original "large chunks are the trigger" hypothesis: the largest single chunk in the corrupted range was 2,832 characters, well inside the safe zone. **The operative quantity is the batch total** — Lumen sends 32 chunks per request, and the corrupting requests carried 16,698–31,364 characters each.

What follows from it, in the code

- `scripts/lumen-index-doctor.sh` decodes the vector store and checks **aggregate distinctness** alongside the conventional per-vector tests. Exit `1` means corruption found.
- `scripts/ollama-vulkan-remediation.sh` and `scripts/lumen-reindex.sh` both probe with a **32-text batch** and require 32 distinct vectors back — a single-vector probe cannot see mode 4.
- `scripts/lumen-reindex.sh` **refuses to start** on a `library=Vulkan` backend rather than writing more of it.
- Tests `I7` and `I8` exist to stop the distinctness check being quietly removed again.

The backend is fixed; check your index anyway

`GGML_VK_VISIBLE_DEVICES=-1` in `/etc/sysconfig/ollama` plus a restart put ollama on `library=cpu`; there have been **0 i915 faults since**, and a 32-chunk / 18,576-character batch returns 32 distinct vectors 5 runs out of 5. `OLLAMA_VULKAN=false` and `OLLAMA_LLM_LIBRARY=cpu` were both tested and are **no-ops**.

Fixing the backend repairs nothing already written. A corrupted file keeps a valid content hash, so an incremental re-index skips it *forever*:

```
./scripts/lumen-index-doctor.sh "$PWD"            # exit 1 =>
        rebuild required
./scripts/lumen-reindex.sh "$PWD" --force         # --force is
        mandatory here
./scripts/lumen-index-doctor.sh "$PWD"            # exit 0 =>
        clean
```

Details: OPERATIONAL-SCRIPTS.md · OLLAMA-REMEDIATION.md.

How It Handles Existing Installations

- **Idempotent**: Re-running does not duplicate configurations or re-clone repositories unnecessarily.
- **Backups**: Every file the wizard touches (agent configs, `~/.claude/settings.json`, `~/.bashrc`, `~/.bash_profile`, the `lumen` wrapper) is copied to a timestamped sibling **and** recorded in a rollback manifest before modification — see Safety and reversibility.
- **Managed blocks**: Three marker-delimited snippets are written — the Lumen block and the telemetry opt-out block in `~/.bashrc`, and the user-bin block in `~/.bash_profile`. Each is stripped and rewritten in place, so three runs still leave exactly one copy of each and your own content is untouched.
- **Skip logic**:
  - If a CLI is already on `PATH`, it is **not** re-installed and the existing install is left alone.
  - If a repository (SuperSpec, Claude plugins) is already cloned, it skips the clone.
  - If a config key already exists, `jq` merges and de-duplicates it safely.
  - A registered SuperSpec **submodule is never deleted** – see What Changed and Why.

Safety and reversibility

Every mutation the wizard performs is recorded in a **transactional manifest**, and there is a companion tool that replays it in reverse.

The wizard prints the session path before it does anything:

```
    Backup session: /home/you/.local/share/setup-agents-wizard/
backups/20260826T210411Z
    Undo everything later with: /path/to/repo/scripts/rollback-
agents-wizard.sh
```

Each session holds a `manifest.tsv` (`component`, `action`, `target`, `backup`, `sha256_before`, `timestamp`) and a `files/` directory of byte-exact copies made with `cp -p`. A `latest` symlink points at the newest session, and `WIZARD_STATE_DIR` overrides the location.

```
./scripts/rollback-agents-wizard.sh --list
        # which runs can be undone
./scripts/rollback-agents-wizard.sh --dry-run
        # print the plan, change nothing
./scripts/rollback-agents-wizard.sh --yes                  # undo
        the newest run
./scripts/rollback-agents-wizard.sh -c shell --yes         # undo
        just the PATH changes
./scripts/rollback-agents-wizard.sh -c opencode --yes      #
        remove Lumen from Opencode only
./scripts/rollback-agents-wizard.sh --run-actions --yes    # also
        undo npm installs, SpecKit and the claude mcp entry
```

- `MODIFIED` entries are restored byte-exactly, permissions included; `CREATED` entries are deleted — including a config the wizard had to create from scratch, because the snapshot is taken *before* the `{}` seed; `ACTION` entries (an `npm install -g`, `uv tool install specify-cli`, the `claude mcp` registration) are **printed** and executed only with `--run-actions`.
- A file touched by several steps still restores to its **true** original — the first snapshot in a session wins.
- **The rollback is itself reversible**: current state is copied into `<session>/pre-rollback-<UTC>/` before anything is touched.
- Exit codes: `0` success, `1` failures occurred, `2` bad option.

Full reference — manifest format, every flag, per-component recipes, how to reverse a rollback, and a manual uninstall for each component if the manifest is lost: `SAFETY-AND-ROLLBACK.md`.

Post-Setup Verification

After the script finishes, it prints **six** summary sections:

1. `Installed Global Commands` — / for `git`, `node`, `npm`, `bun`, `lumen`, `codegraph`, `glyphdown`, `specify`, `kimi`, `opencode`, `mimo`, `qwen`
2. `Claude Code plugins / optional tools` — the **ashlr plugin** (by directory, never by `command -v`) and WOZCODE
3. `Agent MCP Configurations (Lumen)` — one line per agent. Every line probes **behaviour or content**, never mere file existence: `claude mcp get lumen`, `jq -e '.mcpServers.lumen'`, `jq -e '.mcp.lumen'`, `mimo mcp list`. `n/a` means the agent is not installed

4. `Lumen Semantic Search` — launcher (does it answer `version`?), completions (do they register `__start_lumen`?), the `~/.bashrc` and `~/.bash_profile` blocks, `ollama`, the embedding model
5. `Telemetry / analytics` — the `~/.bashrc` opt-out block, CodeGraph, Qwen usage statistics, plus a **live `strings` probe** of the Lumen binary rather than a restated past finding
6. `Project` — SpecKit (non-empty `.specify/` or `specs/`), SuperSpec, the Glyphdown hook

…then the **ACTION REQUIRED** section, and on a terminal a pause for Enter.

Manual next steps. The authoritative list is the one the wizard prints for *your* machine, and it is saved to `MANUAL-STEPS.md`. In general:

- **Claude Code**: restart the IDE extension. If the ashlr plugin was installed, run the three slash commands the wizard names (`/plugin marketplace add ashlrai/ashlr-plugin`, `/plugin install ashlr@ashlr-marketplace`, `/reload-plugins`) and confirm with `/ashlr:ashlr-status`. Optionally `/ashlr:ashlr-genome-init` to build `.ashlrcode/genome/`.

- **Opencode**: simply run `opencode` – Lumen is auto-loaded from `~/.config/opencode/opencode.json`.

- **Kimi** / **Qwen**: run their respective CLIs. They pick up the MCP servers from `~/.kimi-code/mcp.json` and `~/.qwen/settings.json`.

- **MiMo Code**: nothing to do. It inherits Lumen from `~/.claude.json`; check with `mimo mcp list | grep lumen`.

- **All agents**: index this project once, then search it:

  ```
  ./scripts/lumen-reindex.sh "$PWD"
  ./scripts/lumen-index-doctor.sh "$PWD"     # exit 0 = the
          vectors are trustworthy
  lumen search "how does X work" -p "$PWD"
  ```

  Indexing is incremental – re-run it after large external edits. Use the doctor rather than assuming a successful index means a *correct* one; see Why “no NaN” did not mean “no corruption”.

  > An earlier revision of this section told you to run `/cost-mode` and `/fixclaude` and claimed those were “the ones the wizard itself prints”. **The wizard prints neither.** It does print `/reload-plugins`, which that revision said it did not.

Testing and Verification

The wizard has a companion test suite:

```
./scripts/test-setup-agents-wizard.sh            # run everything
./scripts/test-setup-agents-wizard.sh --no-live  # skip the
        network/daemon checks
```

It runs assertions across **eleven groups, A–K**. Note that the file declares group `H` *before* group `G`, so the console order is A, B, C, D, E, F, H, G, I, J, K:

Group	Coverage
A	Static analysis of the wizard (`A1`–`A50`) – `bash -n`, no executable line installing a bogus package (`@qoomon/lumen`, `@monday/codegraph`, `@specify/cli`, …), the real package names (`@colbymchenry/codegraph`, `@qwen-code/qwen-code`, `uv tool install specify-cli`), `stdio` not `serve`, absolute MCP command, the `-e` gitlink test, the `qwen` (not `qwen-code`) probe name, valid `PreToolUse`/`PostToolUse` hook events with no `ToolCall`, the `WIZARD_SKIP_GLYPHDOWN_HOOK` and `WIZARD_KEEP_TELEMETRY` opt-outs, `codegraph sync` and never `codegraph index`, the `speckit` undo action, the `/api/embed` backend health round-trip, library-mode guard, sequential `Step 1..9` headers, `shellcheck -S error`, and (`A58`) that the wizard pins **no** per-host ollama concurrency value. **`A35`–`A41`** cover the GPU/Vulkan detection: `/api/ps` residency, the `inference compute` log line, an unreadable library reported as *unknown*, and — `A41` — that the wizard **never** applies the remediation itself (no `sudo`, no

Group	Coverage
	service restart, no write under `/etc`). **`A42`–`A50`** cover the newest work: ashlr verified by plugin directory rather than `command -v`, the `~/.claude.json` mirror, MiMo probed with `mimo mcp list`, every `lumen search` probe `timeout`-bounded, the `ACTION REQUIRED` section, `MANUAL-STEPS.md`, and the `plugins/marketplaces/` activation probe.
B	Lumen launcher unit tests – executability, `sort -V` version selection, `~/.claude*` fallback, `LUMEN_BIN` override, exit `127` (never a silent success) when the binary is missing.
C	Shell-config unit tests – exactly one managed block per file, idempotence across three runs, generated files parse as bash, the `$PATH` guard written unexpanded, `~/.local/bin` added exactly once, file mode `600` preserved, pre-existing content preserved.
D	MCP-config unit tests – `args == ["stdio"]`, absolute command, valid JSON, foreign keys preserved, exactly two servers after two runs.
E	SuperSpec submodule **data-loss regression** – a gitlink submodule with a canary file survives; a stale non-git directory is still replaced.
F	Live integration – `lumen` on `PATH` in login / login+interactive / interactive shells, embedding backend reachable, model present, completion registration, and a true end-to-end

Group	Coverage
	index-then-search against a two-file fixture repository.
G	Backup manifest and rollback (`G1`–`G18`) – the rollback script parses; the manifest has a TSV header; a pre-existing file is recorded `MODIFIED` and a new one `CREATED`; the stored copy holds the **original** content and mode `600`; re-snapshotting keeps only the first; rollback restores byte-exactly and deletes what was created; `--dry-run` changes nothing; `--component` rolls back only what was selected; a pre-rollback snapshot is written; an `ACTION` is reported but never executed without `--run-actions`; `--list` enumerates sessions. `G16`–`G18` are the regression guards for the snapshot ordering: a brand-new agent config is recorded `CREATED` (not `MODIFIED`) and rollback **deletes** it instead of leaving an empty `{}`, while a pre-existing config still restores to its true original.
H	Telemetry opt-out – exactly one `# >>> telemetry opt-out …` block in `~/.bashrc` and still exactly one after a second run; the generated file parses as bash; sourcing it really exports `DO_NOT_TRACK=1` and `CODEGRAPH_TELEMETRY=0`; `WIZARD_KEEP_TELEMETRY` leaves everything untouched; `.usageStatisticsEnabled` is set to `false` in `~/.qwen/settings.json` without losing existing keys; and a value already `false` is read back as `false` rather than “unset” (`H1`–`H7`).

Group	Coverage
I	**The three operational scripts** (`I1`–`I14`) – each is executable and parses; the remediation script offers `--check` and `--rollback` and **defaults to the read-only action**; its probe tests aggregate **distinctness**, not per-vector health; the doctor checks duplicate-vector groups, still runs the NaN / all-zero tests, and opens the database `mode=ro`; the reindexer refuses to start on a Vulkan backend, and `--force` is proven to actually reach `lumen index -f` rather than merely being parsed; neither the reindexer nor the doctor calls `sudo`.
K	**The two delegated probes wired into the wizard** (`K1`–`K22`) – every assertion here RUNS `tune_ollama_step` / `verify_provider_ci_step` against a throwaway stand-in whose output carries a random per-run token, and checks the observable result: what the stand-in was invoked with, what landed in the manual-step registry, and what landed in the rollback manifest. Covers the degradation path (either script absent, `ollama` absent, `gh` absent, a delegate that fails — the wizard continues and says so), the destructive edge (report-only by default; `--apply` refused while an indexer is in flight **and** when that state cannot be determined; the opt-in path proven to actually apply so the refusals are not vacuous), the rollback story (`ACTION … --revert`

Group	Coverage
	plus a `NOTE` for the irreversible restart; rollback prints `NOT UNDONE`), and the verifier's three-valued verdict (`2` reported as UNVERIFIED and **never** as the `0` success line). `K21`/`K22` count the **call sites**, because everything else would stay green if the wiring were deleted.
J	**Hardcoded-path audit** (`J1`–`J8`) – guards `scripts/audit-hardcoded-paths.sh`, which fails the build on machine-specific absolute paths (18 tracked files once hardcoded one author's `/Volumes/…` macOS root). `J2`–`J7` exercise it against **throwaway git repositories** so each verdict is proven rather than asserted: clean repo exits `0`, a machine path exits `1`, a *comment* describing the historical bug does **not** count, `$HOME` and ~ are allowed, `.hardcoded-paths-allow` really suppresses a listed file, and `/etc` `/usr` `/tmp` are not treated as machine-specific. `J8` turns the rule on this repository's own scripts.

`--no-live` skips group F, so the suite runs with no network access and no `ollama` daemon.

Groups B, C, D, G and H run inside a **throwaway `$HOME`** (via the wizard's `SETUP_WIZARD_LIB_ONLY=1` library mode, with `WIZARD_STATE_DIR` pointed inside it), and groups E and J inside throwaway git repositories – **the real environment is never touched**. Groups A and I only read source text.

Do not hardcode a total. The count moves whenever a test is added, and it differs between a live run and `--no-live`. Derive it from the per-group record counts: `A`=54 (50 ids — `A12` loops over 5 bogus package names), `B`=8, `C`=10, `D`=5, `E`=3, `F`=11 live / 7 with `--no-live` (`F2`

loops over 3 shell kinds), `G`=18, `H`=7, `I`=14 (11 assertions plus a 3-script parse loop), `J`=8. That is **138** for a full live run and **134** with `--no-live`. `summary.json` prints the authoritative `{total, passed, failed, skipped}` for the run you actually did — quote that, not a number from this table.

Every assertion records machine-readable evidence – expected value, actual value, status and timestamp – under `.test-evidence/<UTC timestamp>/`:

File	Contents
`results.tsv`	One row per assertion: `id`, `group`, `name`, `status`, `expected`, `actual`, `timestamp`.
`run.log`	Full console transcript of the run.
`summary.json`	Host details and `{total, passed, failed, skipped}` plus the exit code.

Exit status is non-zero if any assertion failed.

Troubleshooting

Symptom	Likely Cause	Fix
`lumen: command not found`	`~/.local/bin` not yet on `PATH` in this shell	Open a new terminal, or run `export PATH="$HOME/.local/bin:$PATH"`.
`lumen` works in your terminal but not in cron / `ssh host '<cmd>'` / a systemd unit	`~/.bashrc` returns early for non-interactive shells	The wizard writes the `~/.local/bin` guard into `~/.bash_profile` for exactly this reason – re-run it, or add the guard by hand.
`Error: unknown command "serve" for "lumen"`	Stale MCP config from an older revision of this wizard	Re-run the wizard, or set `"args": ["stdio"]` in the agent's config.

Symptom	Likely Cause	Fix
`lumen: could not locate the Lumen plugin binary` (exit 127)	The Claude Code Lumen plugin is not installed	Install it from `/plugin` inside Claude Code, or `export LUMEN_BIN=/path/to/lumen-<os>-<arch>`.
Indexing or search fails; backend unreachable	`ollama` not running, or the embedding model was never pulled	`sudo systemctl enable --now ollama`, then `ollama pull ordis/jina-embeddings-v2-base-code`.
`Embedding backend is WEDGED … it returns NaN for every input`	The ollama runner is up (`/api/tags` still returns 200) but embedding is broken — this is why Step 3 does a real `/api/embed` round-trip instead of a ping	Run the fix the wizard prints: `ollama stop ordis/jina-embeddings-v2-base-code`, then re-index — TROUBLESHOOTING.md
`Embedding model is GPU-offloaded and ollama reports library=Vulkan`	The corrupting path. It writes stale duplicate vectors under HTTP 200 that no per-vector check can see	`./scripts/ollama-vulkan-remediation.sh --check`, then `--apply` — OPERATIONAL-SCRIPTS.md
Searches “work” but return plausible-looking nonsense, or miss files you know exist	Possible stale-duplicate vectors in the index	`./scripts/lumen-index-doctor.sh "$PWD"` — exit `1` means rebuild with `./scripts/lumen-reindex.sh "$PWD" --force`. Read Why “no NaN” did not mean “no corruption”
`REFUSING TO START: ollama reports library=Vulkan` (exit 3) from `lumen-reindex.sh`	Working as designed — it will not write more corrupt vectors	Fix the backend first, or `--allow-gpu` if you accept the risk
`command -v ashlr` finds nothing		Check `~/.claude/plugins/cache/ashlr-`

Symptom	Likely Cause	Fix
	Expected. ashlr is a Claude Code plugin with no binary	`marketplace/ashlr/` instead, then run the three slash commands — `ACTION-REQUIRED.md`
`mimo mcp list` does not show `lumen`	`~/.claude.json` has no `.mcpServers.lumen`, or MiMo was started before it was added	Re-run the wizard (it mirrors the entry into `~/.claude.json`), then restart MiMo
Kimi / Opencode / MiMo Code not installed	They are detected, never npm-installed – the bare npm names are unrelated packages	Install from the vendor: https://kimi.moonshot.cn, https://opencode.ai, https://github.com/XiaomiMiMo, then re-run the wizard so it can write their MCP config.
MCP servers not showing in agent	Agent was configured at an old, wrong path by a previous revision	Re-run the wizard; it writes the verified paths listed in MCP Configuration for Every Agent. Delete leftovers like `~/.claude/mcp.json` or `~/.qwen-code/config.json` – nothing reads them.
`jq` installation fails	Unsupported OS or missing package manager	Install `jq` manually from https://stedolan.github.io/jq/
SuperSpec extension fails	`specify` CLI not in PATH	Install SpecKit with `uv tool install specify-cli`, then run `specify extension add ./submodules/superspec --dev` manually.
WOZCODE skipped	`WOZCODE_INSTALL_CMD` not set	Register at wozcode.com, set the

Symptom	Likely Cause	Fix
		env var, and re-run the script.
You want the whole run undone	–	`./scripts/rollback-agents-wizard.sh --dry-run`, then `--yes` — SAFETY-AND-ROLLBACK.md
`No backup sessions found at …` from the rollback tool	The wizard never ran, or `WIZARD_STATE_DIR` differs between the two	Point both at the same directory, or uninstall by hand — SAFETY-AND-ROLLBACK.md

For the long-form, symptom-first version of this table – including a 60-second health check and how to collect evidence for a bug report – see TROUBLESHOOTING.md.

Resetting / Uninstalling

The script **does not** modify your project source code (only adds configs).

Preferred: replay the manifest. The wizard recorded everything it changed, so let the rollback tool undo it rather than deleting things by hand:

```
./scripts/rollback-agents-wizard.sh --list                # pick
        a session
./scripts/rollback-agents-wizard.sh --dry-run             # read
        the plan
./scripts/rollback-agents-wizard.sh --run-actions --yes   #
        apply, including npm/claude undo commands
exec bash -l
```

That restores every modified file byte-exactly, deletes every file the wizard created, and keeps a `pre-rollback-<UTC>` snapshot so the undo is itself undoable. Details and per-component recipes: SAFETY-AND-ROLLBACK.md.

If the manifest is gone, remove things by hand, skipping anything you still want:

1. **Global packages the wizard installs** (there is no Lumen npm package – it was never installed from npm, and Kimi/Opencode/MiMo are never npm-installed either):

   ```
   npm uninstall -g @colbymchenry/codegraph glyphdown @qwen-code/
           qwen-code
   uv tool uninstall specify-cli
   ```

2. **Remove the Lumen launcher and completions** (optional):

   ```
   rm -f ~/.local/bin/lumen ~/.local/bin/lumen.bak.*
   rm -f ~/.local/share/bash-completion/completions/lumen
   ```

3. **Remove the Lumen MCP entries** – per agent, not by deleting whole config directories:

   ```
   claude mcp remove lumen -s user
   jq 'del(.mcpServers.lumen)'
           ~/.claude.json                  # the mirror MiMo
           inherits
   jq 'del(.mcpServers.lumen, .mcpServers.codegraph)' ~/.kimi-
           code/mcp.json           # then move into place
   jq 'del(.mcp.lumen)'                                    ~/.config/
           opencode/opencode.json
   jq 'del(.mcpServers.lumen, .mcpServers.codegraph)' ~/.qwen/
           settings.json
   ```

4. **Clean project**:

   ```
   rm -rf submodules/ .specify/ specs/
   rm -f MANUAL-STEPS.md .lumen-reindex.log
   ```

5. **Undo the ollama Vulkan remediation**, if you applied it and want the GPU back (it restores the defect):

   ```
   ./scripts/ollama-vulkan-remediation.sh --rollback
   ```

   It is **not** in the rollback manifest — `scripts/rollback-agents-wizard.sh` will not touch it.

The managed `~/.bashrc` / `~/.bash_profile` blocks are best removed with the wizard's own `strip_managed_block` helper rather than by hand. `SAFETY-AND-ROLLBACK.md` and `FAQ.md` have the complete, ordered procedure.

Script Source

`scripts/setup-agents-wizard.sh` is the single source of truth. Read it there:

```
less scripts/setup-agents-wizard.sh
```

For a narrated tour of what it does, use `MANUAL.md` (function-by-function reference) and `ARCHITECTURE.md` (flow diagrams).

> **A previous revision of this README embedded a full copy of the script.** That copy is gone on purpose: a duplicated 650-line listing in documentation always drifts out of date, and this one had. It contained a non-existent npm package (`@qoomon/lumen`), an MCP config using the non-existent `serve` subcommand, and – worst – a SuperSpec check that **deleted a registered git submodule on every run**. If you or anyone on your team copied that listing into a local file, **discard it** and use `scripts/setup-agents-wizard.sh`.

What Changed and Why

Change	Why
Removed `npm install -g @qoomon/lumen`	That package does not exist – the npm registry returns 404. The `|| true` on the install line swallowed the failure silently, so `lumen` stayed missing while the summary still looked healthy. Lumen ships as a Claude Code plugin binary and is now set up in Step 3.
MCP `"args"` changed from `["serve"]` to `["stdio"]`	Lumen has no `serve` subcommand; it errors with `unknown command "serve" for "lumen"`. `stdio` is the real MCP transport subcommand.
MCP `"command"` is now the absolute path to `~/.local/bin/lumen`	Agents spawn MCP servers from non-interactive shells, where `~/.local/bin` is frequently absent from `PATH`. A bare `"lumen"` would fail to launch.

Change	Why
SuperSpec check `[[ -d "$path/.git" ]]` → `[[ -e "$path/.git" ]]`	**Data loss.** For a registered git submodule, `<path>/.git` is a **file** (a gitlink), never a directory – so the `-d` test was always false and the `rm -rf` beneath it deleted the registered submodule on **every single run**, replacing it with a plain clone and corrupting the parent repository's submodule state. The logic now lives in `setup_superspec()`, tests with `-e`, initializes registered-but-missing submodules via `git submodule update --init --recursive`, and only removes a directory that git does not track as a submodule. Test group **E** is the regression guard.
New Step 3: Lumen Semantic Search Setup	Installs the version-agnostic `~/.local/bin/lumen` wrapper (resolved with `sort -V`, since a lexical sort ranks `0.0.9` above `0.0.100`), bash completions, the PATH blocks, Ollama and the embedding model – then verifies all of it. The step count went from 5 to 6; every later step shifted by one. It later grew to 7 with the opt-in indexing step.
New: telemetry opt-out	Analytics were left on by default for tools the wizard installed. `configure_telemetry_optout` now runs first inside Step 3: a third managed `~/.bashrc` block exporting `DO_NOT_TRACK=1` and `CODEGRAPH_TELEMETRY=0`, `codegraph telemetry off`, and `.usageStatisticsEnabled = false` in `~/.qwen/settings.json`. `WIZARD_KEEP_TELEMETRY=1` skips all of it. Lumen was checked and ships no telemetry, so nothing is

Change	Why
	claimed for it. Test group **H** is the guard. See Telemetry Opt-Out.
New Step 8: opt-in project indexing	Installing the indexers never indexed anything, so a fresh machine had a working `lumen` and an empty index. `WIZARD_INDEX_PROJECT=1` now runs `codegraph sync`/`codegraph init` and `lumen index` for the project root. It is opt-in because a first Lumen pass on a large repository can run for hours. `codegraph index` is deliberately never used — it discards the existing database. See Project Indexing.
Snapshot taken before the `{}` seed	`configure_mcp_for_agent` and the Glyphdown hook step used to write `{}` into a missing config *before* calling `backup_file`, so a file the wizard created from scratch was recorded `MODIFIED` with `{}` as its "original" — and rollback left an empty `{}` behind instead of removing it. `snapshot_before` now runs first, the row is `CREATED`, and rollback deletes the file. Tests **G16**/**G17** are the guards.
`uv tool install specify-cli` now records an undo action	It was the one install with no manifest row. It now writes `ACTION uv tool uninstall specify-cli` under component `speckit`, so `--run-actions` really does undo everything the wizard installed. Test **A28** is the guard.
Login-shell PATH fix in `~/.bash_profile`	`~/.bashrc` – where `~/.local/bin` is normally added – **returns early for non-interactive shells**, and login shells start from the `PATH` set by `/etc/profile`. Without a guard in `~/.bash_profile`, `bash`

Change	Why
	`-lc`, `ssh host '<cmd>'`, cron jobs and systemd user units never see `~/.local/bin` at all. The guard is idempotent: a no-op for interactive shells, the fix for everything else.
Removed the embedded script listing	See Script Source above.
Added `SETUP_WIZARD_LIB_ONLY` library mode + test suite	Lets `scripts/test-setup-agents-wizard.sh` source the wizard's helper functions without running the wizard, so unit tests can exercise them against a throwaway `$HOME`.
New: transactional backup manifest + `scripts/rollback-agents-wizard.sh`	Timestamped `.bak` siblings told you a file *had* changed but not what to do about it, and non-file changes (an `npm install -g`, a `claude mcp` registration) left no trace at all. Every mutation is now recorded under `~/.local/share/setup-agents-wizard/backups/<UTC>/` as `MODIFIED` / `CREATED` / `ACTION`, and the rollback tool replays it — wholesale or one component at a time, dry-runnable, and itself reversible. Test group **G** is the guard. See Safety and reversibility.
Step 2 package names corrected	The wizard installed by *guessed* name. `@qoomon/lumen`, `@monday/codegraph`, `@specify/cli` and `opencode` all return 404, while bare `kimi` and `mimo` resolve to **completely unrelated** packages — a state-animation library and a mobile build tool — which the old `|| true` then reported as success. It now installs only what is missing, by verified name (`@colbymchenry/codegraph`,

Change	Why
	`glyphdown`, `@qwen-code/qwen-code`, and `specify` via `uv tool install specify-cli`), and detects Kimi / Opencode / MiMo rather than installing something that merely shares their name.
Agent configs are extended at their REAL locations	`~/.claude/mcp.json`, `~/.kimi/config.json`, `~/.opencode/config.json`, `~/.mimo/config.json` and `~/.qwen-code/config.json` are not read for these agents — the wizard was writing **orphan files no agent reads**, and ticking them green. Claude Code is now registered through `claude mcp add-json … -s user` rather than by hand-editing the whole file, which also holds session state — only the single `.mcpServers.lumen` key is `jq`-merged into `~/.claude.json`, for the tools that inherit from it; Kimi uses `~/.kimi-code/mcp.json`, Qwen Code `~/.qwen/settings.json`, and Opencode `~/.config/opencode/opencode.json` with its `.mcp` schema. *(This row originally ended by saying MiMo Code had no documented MCP config file. It does not need one — see the row below.)*
Glyphdown hook fixed	`ToolCall` is not a Claude Code hook event, so the old entry was written into `~/.claude/settings.json` and silently ignored. The wizard now writes valid `PreToolUse` and `PostToolUse` entries (`{matcher, hooks:[{type, command}]}`), adds them only when `glyphdown` is actually installed — a hook for a missing

Change	Why
	binary would fire and fail on every tool call — and preserves any hooks you already had.
New: the `ACTION REQUIRED` / manual-steps system	The wizard could not run Claude Code slash commands or use your `sudo`, and silently skipping those steps meant a green summary for work nobody had done. It now **detects** each pending step, prints them under `ACTION REQUIRED`, writes `<project-root>/MANUAL-STEPS.md`, and pauses for Enter on a terminal (`WIZARD_NONINTERACTIVE=1` skips the pause). Detection is stateful: `plugins/cache/<mkt>/` means the installer cloned it, `plugins/marketplaces/<mkt>/` means *you* ran `/plugin marketplace add`. Tests **A48**–**A50**. See `ACTION-REQUIRED.md`.
`ashlr` is no longer probed as a CLI	ashlr is a Claude Code **plugin**; its installer creates no binary, so `check_command ashlr` could never succeed and printed a permanent, misleading . The wizard now verifies `~/.claude/plugins/cache/ashlr-marketplace/ashlr/`, reports it in its own summary section, and surfaces the three slash commands that actually activate it. Tests **A42**, **A43**.
Lumen mirrored into `~/.claude.json`	`claude mcp add-json -s user` writes into the **active `CLAUDE_CONFIG_DIR`**, which is not necessarily `~/.claude.json`. Tools that inherit from the default file — MiMo labels its servers `claude:~/.claude.json` — were left blind to Lumen. The wizard

Change	Why
	now `jq`-merges the entry into `~/.claude.json` as well. Test **A44**.
The “MiMo has no documented MCP config” claim was false	MiMo Code inherits MCP servers from `~/.claude.json`; there was never a manual step. The wizard now **probes** with `timeout 25 mimo mcp list` instead of asserting either way. Tests **A45**, **A46**.
Every `lumen search` probe is `timeout`-bounded	`lumen search` blocks while an index run holds the embedding backend, and an unbounded probe once stalled the wizard for ten minutes. A timeout is now treated as *“could not confirm”* and raises the manual step rather than silently skipping it. Test **A47**.
New: three operational scripts	`ollama-vulkan-remediation.sh`, `lumen-reindex.sh` and `lumen-index-doctor.sh`. Everything privileged lives there, behind an explicit subcommand, so the wizard stays read-only in the embedding path (test **A41**). Test group **I** covers all three. See `OPERATIONAL-SCRIPTS.md`.
The index integrity verdict was overturned	`LUMEN-INDEX-INTEGRITY.md` concluded the index was `TRUSTWORTHY` on the strength of NaN / Inf / all-zero / L2-norm checks. All four are **per-vector**, and all four are blind to a stale-but-well-formed vector — of which the index held 758, across 55 files. The settled verdict is `INDEX-CORRUPTION-RECONCILIATION.md`; the earlier report is kept unedited as a record. See Why “no NaN” did not mean “no corruption”.

Final Notes

This wizard is engineered to be **risk-free, transparent, and fully automated**. It:

- Backs up every configuration file before changing it, and records the change in a manifest you can replay in reverse.
- Installs only what is missing, by verified package name, without forcing risky updates.
- Never overwrites your existing project files. The paths it adds inside your project are `submodules/`, `.specify//specs/`, `MANUAL-STEPS.md`, and — with `WIZARD_INDEX_PROJECT=1` — `.codegraph/`. (`scripts/lumen-reindex.sh` separately writes `.lumen-reindex.log`, and the test suite writes `.test-evidence/`.)
- Never deletes a path that git tracks as a submodule.
- Never invents a config path, and never claims a check it did not run: `n/a` for an agent that is not installed, `UNKNOWN` for a backend library it could not read.
- Never uses `sudo` in the embedding path and never restarts a service. Anything privileged is a separate script behind an explicit subcommand — test **A41**.
- Never reports as done the things it cannot do. They go under `ACTION REQUIRED`, in `MANUAL-STEPS.md`, and on a terminal the run stops until you have read them.

After running it, every agent the wizard could configure shares the same high-performance MCP backbone, giving you **maximum efficiency and cost savings** regardless of which agent you choose to use. And if you change your mind, one command puts it all back: `SAFETY-AND-ROLLBACK.md`.